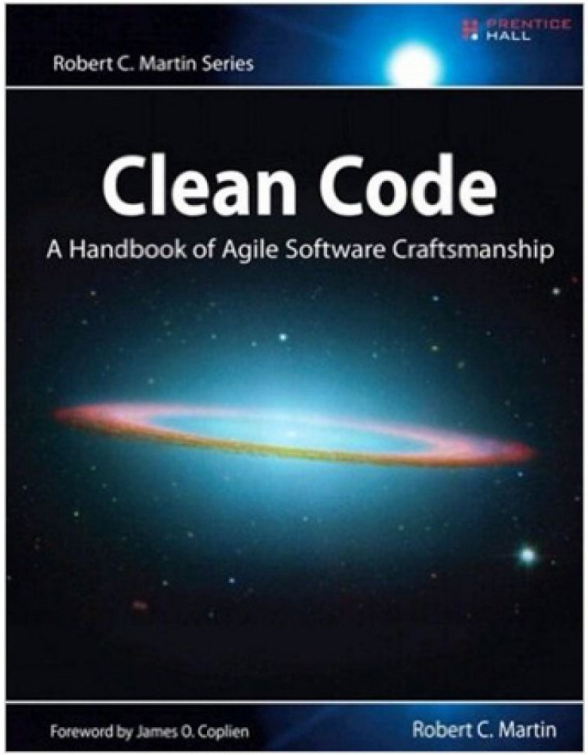
Laborator

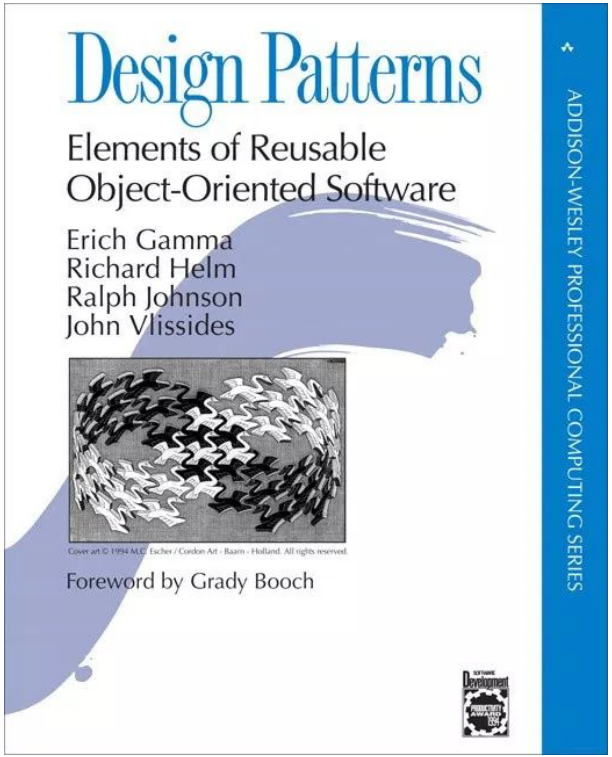
PDSS

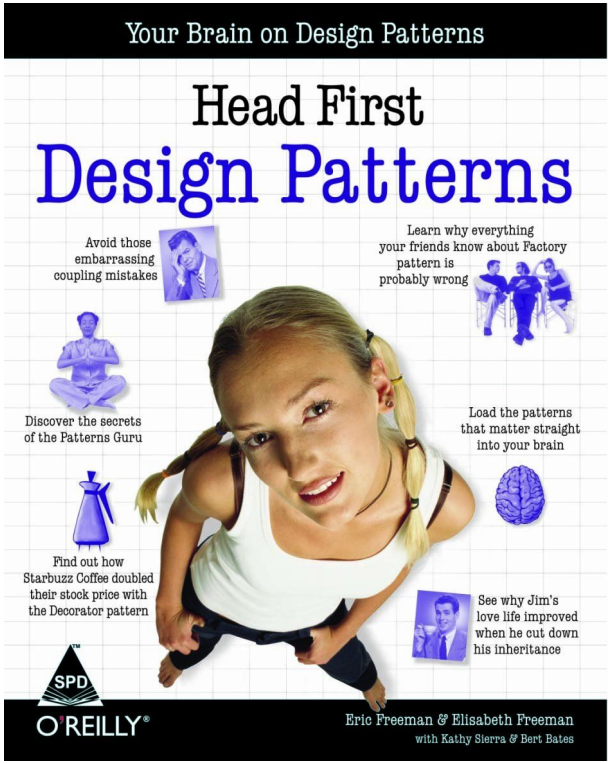
Subiectele laboratorului:

1. Clean Code
2. Refactorings
3. Java Streams
4. Code Smells
5. S.O.L.I.D. Principles
6. Law of Demeter
7. Design Patterns

Sugestii:


PRENTICE HALL
Robert C. Martin Series
Clean Code
A Handbook of Agile Software Craftsmanship
Foreword by James O. Coplien
Robert C. Martin


Design Patterns
Elements of Reusable Object-Oriented Software
Erich Gamma
Richard Helm
Ralph Johnson
John Vlissides
Foreword by Grady Booch
ADDISON-WESLEY PROFESSIONAL COMPUTING SERIES


Your Brain on Design Patterns
Head First
Design Patterns
Avoid those embarrassing coupling mistakes
Learn why everything your friends know about Factory pattern is probably wrong
Discover the secrets of the Patterns Guru
Load the patterns that matter straight into your brain
Find out how Starbuzz Coffee doubled their stock price with the Decorator pattern
See why Jim's love life improved when he cut down his inheritance
SPD
O'REILLY
Eric Freeman & Elisabeth Freeman
with Kathy Sierra & Bert Bates

Clean code

"Always leave the code you are editing better than you found it."

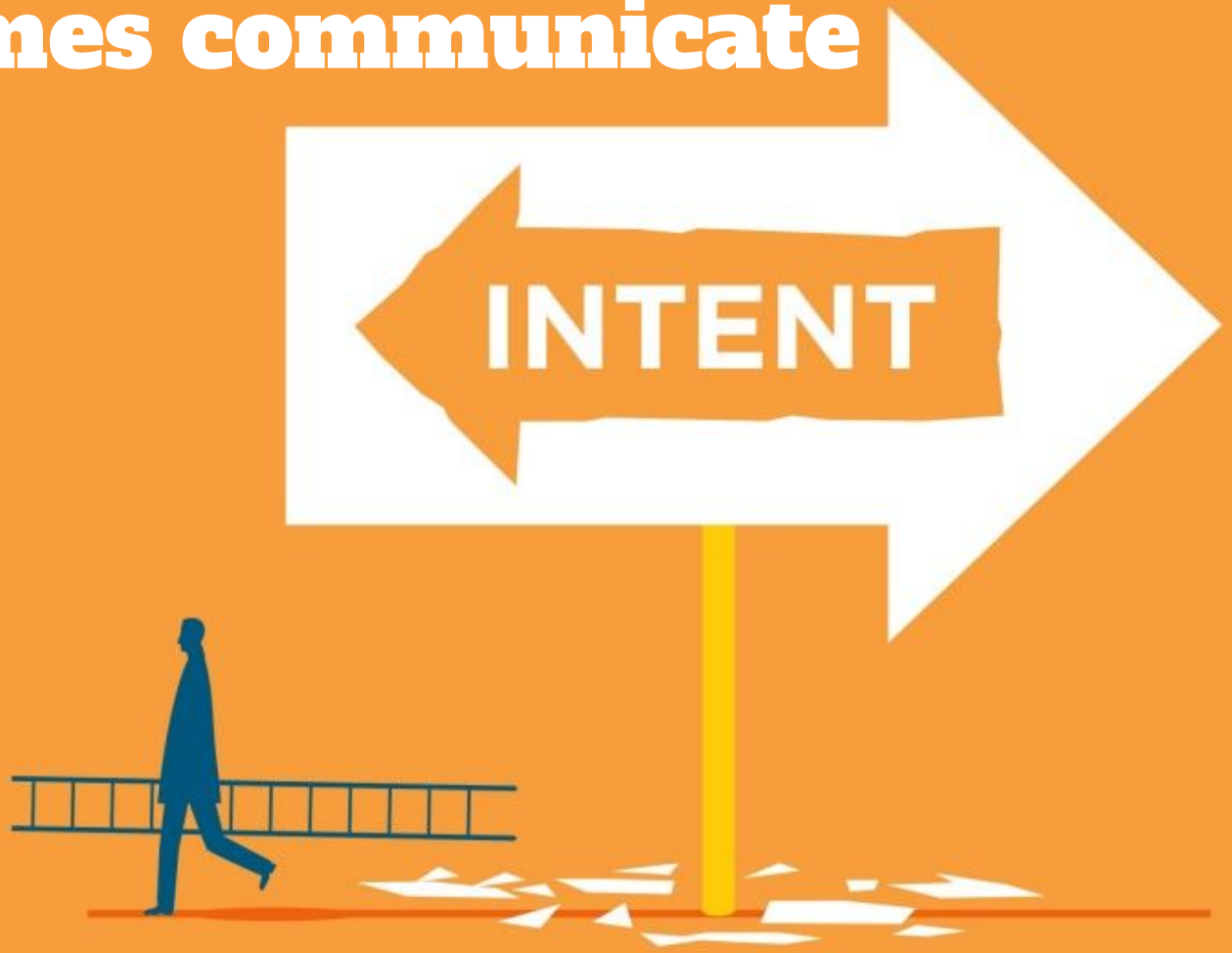
The Boy Scout Rule

Robert C. Martin

Uncle Bob

Names

Names communicate



Names - meaningful

```
int d;

int dd; //days past since project start

int m; //minutes
```

```
int daysRemaining;

int daysSinceProjectStart;

int elapsedMinutes;
```

Names - pronounceable

```
class DfKndSrv {
   private String ord_num;
   private int prq;
   public String descOrdDet() {
      return String.format("Order Number: %s,
product Quantity: %s\n", ord_num, prq);
   }
}
```

```
class DefaultCustomerService {
   private String orderNumber;
   private int productQuantity;

   public String describeOrder() {
      return String.format("Order Number: %s, " +
            "product Quantity: %s\n",
            orderNumber, productQuantity);
   }
}
```

Names - avoid encodings

```
int i_days;

Address addressString;

//type changed but name did not!

String s_desc;
```

```
int days;

Address address;

String description;
```

Scope rule - variables

Short scoped variables should have short names:

```
for (String n : names) {
  System.out.println(n.length() + "." + n);
}
```

Public scoped variables should have long, self-explanatory names:

```
public static final String PROJECT_PLUGINS_PATH = "path/to/plugins";
```

Scope rule - methods

Private methods should have long, descriptive names:

```
private ArrayList<Person> keepOnlyOveragePeople(List<Person> people)
private Student findStudentByNameAndAgeGreaterThan25AndGenderMale(String name)
```

Public methods, oftenly used, should have short names:

```
public interface ProjectRepository {
  Project search(String projectID);
  Project create(String projectID, String projectPath);
}
```

Parts of speech rule

Classes / Variables should be **nouns** :

```
public class Person { private String firstName; private int age; private Status status;}
```

Booleans should be **predicates / adjectives** :

```
public class Person { private boolean overage; public boolean isOverage() {return overage;}}
```

Methods should be written like **actions** :

```
public Person searchByName(String name), public void createWeatherReport(City city),
public int generateRandomNumber(Range range)
```

Enums should be **adjectives** :

```
enum Level { LOW, MEDIUM, HIGH }, enum ActivityStatus { ACTIVE, PASSIVE, RETIRED}
```

$f(x)$

Functions

Functions should be short

5 - 7

Lines of code

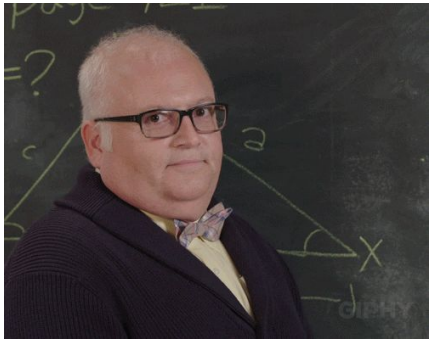
!?
Lost?

Names + do one thing

Function arguments

No Arguments:

1 or 2 Args is OK



3 Arguments...

4+ Arguments...

No output arguments

```
public void computeResult(long x, long y, Result result) {
  //...some code
  result.setResult(x + y);
}
```

```
public Result computeResult(long x, long y) {
  //...some code
  return new Result(x + y);
}
```

No boolean arguments

```
public void doSomething(String someString, boolean up) {
  //...some code
  if (up) {
    //...do some stuff
  } else {
    //...do some other stuff
  }
}
```

2 Things!!!

No boolean arguments

```
public void doSomething(String someString, boolean up) {
  //...some code
  if (up) {
    //...do some stuff
  } else {
    //...do some other stuff
  }
}
```

2 Things!!!

```
public void doSomeStuff(String someString) {
  //...do some stuff
}
```

```
public void doSomeOtherStuff(String someString) {
  //...do some other stuff
}
```

Comments

Usually noisy

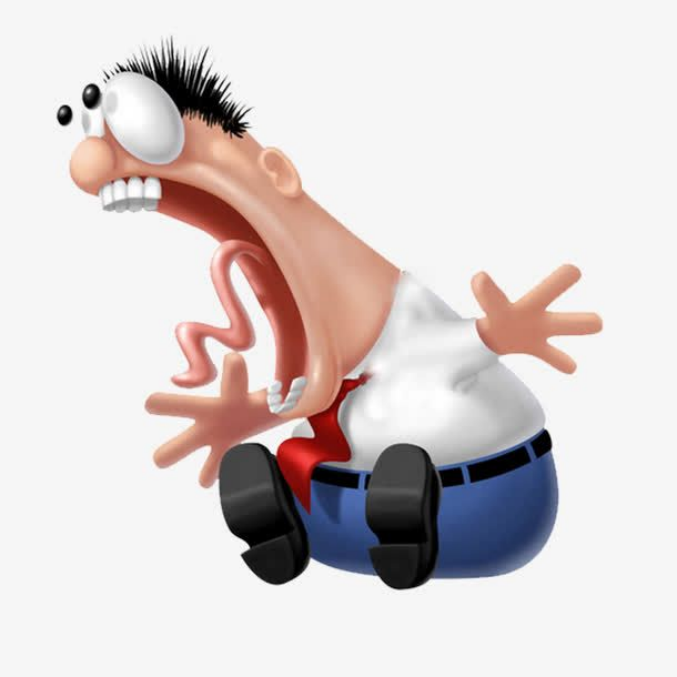
```
/**
* Gets notification.
*
* @param customer customer
* @param id  notification id
* @return the notification for the given customer and notification id.
*/
Optional<NotificationModel> getNotification(CustomerModel customer, String id);
```

Crazy

```
// the name
public static String name;

// the address
public static Address address;

// the phone number
public static Phone phoneNumber;
```

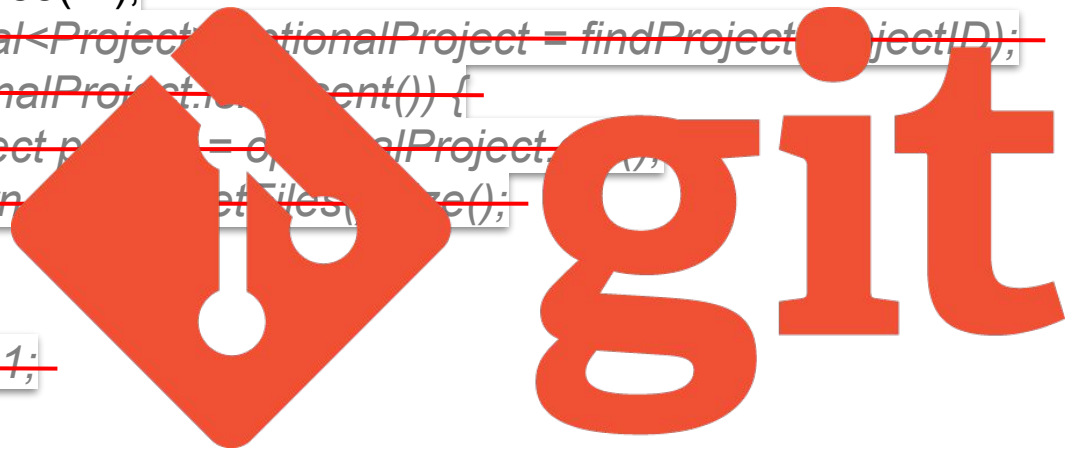


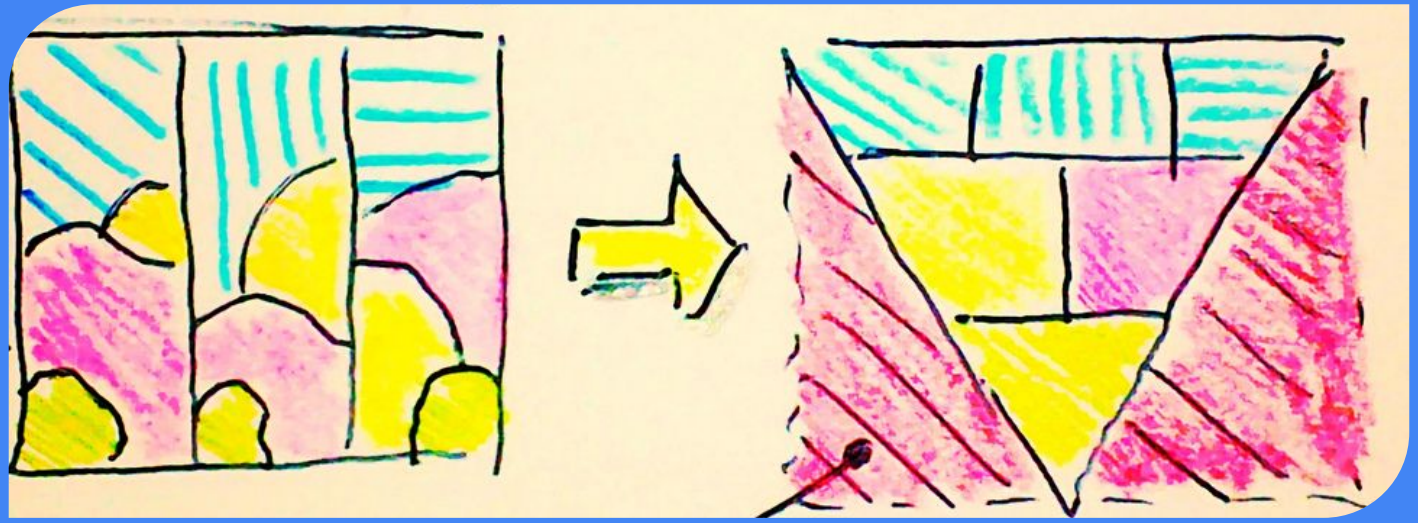
Delete commented Code

```
  public int getNumberOfFilesInProject(String projectID) {
      return findProject(projectID)
              .map(project -> project.getFiles().size())
              .orElse(-1);
//        Optional<Project> optionalProject = findProject(projectID);
//        if(optionalProject.isPresent()) {
//            Project project = optionalProject.get();
//            return project.getFiles().size();
//        }
//
//        return -1;
  }
```

Delete commented Code

```
public int getNumberOfFilesInProject(String projectID) {
    return findProject(projectID)
            .map(project -> project.getFiles().size())
            .orElse(-1);
//          Optional<Project> optionalProject = findProject(projectID);
//          if(optionalProject.isPresent()) {
//              Project p = optionalProject.get();
//              return p.getFiles().size();
//          }
//
//          return -1;
}
```





Refactoring

Example

Why should
we refactor?

```java
public class Person {
   private String firstName;
   private String lastName;
   private int age;
   private Status status;

   //constructor, getters, setters
}
```

```java
public enum Status {
   SINGLE,
   MARRIED,
   DIVORCED,
   WIDOW
}
```

```java
public class Manager {
  public List<String> getPeopleNames(List<Person> people) {
    ArrayList<Person> personList = new ArrayList<>();
    ArrayList<String> stringList = new ArrayList<>();
    for (Person person : people) {
      int x = person.getAge();
      if (x >= 18) {
        personList.add(person);
      }
    }
    for (int i = 0; i < personList.size(); i++) {
      Person p = personList.get(i);
      String s1 = p.getFirstName() + " " + p.getLastName();
      stringList.add(s1);
    }
    for (int i = 0; i < stringList.size(); i++) {
      for (int j = i; j < stringList.size(); j++) {
        if (stringList.get(i).compareToIgnoreCase(stringList.get(j)) > 0) {
          String aux = stringList.get(j);
          stringList.set(j, stringList.get(i));
          stringList.set(i, aux);
        }
      }
    }
    for (String s : stringList) System.out.println(s);
    return stringList;
  }
}
```

```java
public class Person {
  private String firstName;
  private String lastName;
  private int age;
  private Status status;

  //constructor, getters, setters
}
```

```java
public enum Status {
  SINGLE,
  MARRIED,
  DIVORCED,
  WIDOW
}
```

```java
public class Manager {
  public List<String> getPeopleNames(List<Person> people) {
    ArrayList<Person> personList = new ArrayList<>();
    ArrayList<String> stringList = new ArrayList<>();
    for (Person person : people) {
      int x = person.getAge();
      if (x >= 18) {
        personList.add(person);
      }
    }
    for (int i = 0; i < personList.size(); i++) {
      Person p = personList.get(i);
      String s1 = p.getFirstName() + " " + p.getLastName();
      stringList.add(s1);
    }
    for (int i = 0; i < stringList.size(); i++) {
      for (int j = i; j < stringList.size(); j++) {
        if (stringList.get(i).compareToIgnoreCase(stringList.get(j)) > 0) {
          String aux = stringList.get(j);
          stringList.set(j, stringList.get(i));
          stringList.set(i, aux);
        }
      }
    }
    for (String s : stringList) System.out.println(s);
    return stringList;
  }
}
```

```java
public class Person {
  private String firstName;
  private String lastName;
  private int age;
  private Status status;

  //constructor, getters, setters
}
```

```java
public enum Status {
  SINGLE,
  MARRIED,
  DIVORCED,
  WIDOW
}
```

What does the *getPeopleNames()* method do?

```java
public class PersonService {

  public List<String> getOveragePeopleNames(List<Person> people) {

      List<Person> overagePeople = keepOnlyOveragePeople(people);

      List<String> overagePeopleNames = getPeopleFullNames(overagePeople);

      overagePeopleNames = sortNamesAlphabetically(overagePeopleNames);

      printNames(overagePeopleNames);

      return overagePeopleNames;
  }
...
}
```

```java
public class Person {
  private String firstName;
  private String lastName;
  private int age;
  private Status status;

  //constructor, getters, setters
}
```

```java
public enum Status {
  SINGLE,
  MARRIED,
  DIVORCED,
  WIDOW
}
```

What does the *getPeopleNames()* method do?

```java
public class Manager {
  public List<String> getPeopleNames(List<Person> people) {
    ArrayList<Person> personList = new ArrayList<>();
    ArrayList<String> stringList = new ArrayList<>();
    for (Person person : people) {
      int x = person.getAge();
      if (x >= 18) {
        personList.add(person);
      }
    }
    for (int i = 0; i < personList.size(); i++) {
      Person p = personList.get(i);
      String s1 = p.getFirstName() + " " + p.getLastName();
      stringList.add(s1);
    }
    for (int i = 0; i < stringList.size(); i++) {
      for (int j = i; j < stringList.size(); j++) {
        if (stringList.get(i).compareToIgnoreCase(stringList.get(j)) > 0) {
          String aux = stringList.get(j);
          stringList.set(j, stringList.get(i));
          stringList.set(i, aux);
        }
      }
    }
    for (String s : stringList) System.out.println(s);
    return stringList;
  }
}
```

```java
public class Manager {
  public List<String> getPeopleNames(List<Person> people) {
    ArrayList<Person> personList = new ArrayList<>();
    ArrayList<String> stringList = new ArrayList<>();
    for (Person person : people) {
      int x = person.getAge();
      if (x >= 18) {
        personList.add(person);
      }
    }
    for (int i = 0; i < personList.size(); i++) {
      Person p = personList.get(i);
      String s1 = p.getFirstName() + " " + p.getLastName();
      stringList.add(s1);
    }
    for (int i = 0; i < stringList.size(); i++) {
      for (int j = i; j < stringList.size(); j++) {
        if (stringList.get(i).compareToIgnoreCase(stringList.get(j)) > 0) {
          String aux = stringList.get(j);
          stringList.set(j, stringList.get(i));
          stringList.set(i, aux);
        }
      }
    }
    for (String s : stringList) System.out.println(s);
    return stringList;
  }
}
```

```java
public class Manager {
  public List<String> getPeopleNames(List<Person> people) {
    ArrayList<Person> personList = new ArrayList<>();
    ArrayList<String> stringList = new ArrayList<>();
    for (Person person : people) {
      int x = person.getAge();
      if (x >= 18) {
        personList.add(person);
      }
    }
    for (int i = 0; i < personList.size(); i++) {
      Person p = personList.get(i);
      String s1 = p.getFirstName() + " " + p.getLastName();
      stringList.add(s1);
    }
    for (int i = 0; i < stringList.size(); i++) {
      for (int j = i; j < stringList.size(); j++) {
        if (stringList.get(i).compareToIgnoreCase(stringList.get(j)) > 0) {
          String aux = stringList.get(j);
          stringList.set(j, stringList.get(i));
          stringList.set(i, aux);
        }
      }
    }
    for (String s : stringList) System.out.println(s);
    return stringList;
  }
}
```

```java
List<Person> personList =
                    keepOnlyOveragePeople(people);
```

```java
private ArrayList<Person> keepOnlyOveragePeople
                    (List<Person> people) {
  ArrayList<Person> overagePeople = new ArrayList<>();
  for (Person person : people) {
    if (person.getAge() >= 18) {
      overagePeople.add(person);
    }
  }
  return overagePeople;
}
```

```java
public class Manager {
  public List<String> getPeopleNames(List<Person> people) {
    List<Person> personList = keepOnlyOveragePeople(people);


    ArrayList<String> stringList = new ArrayList<>();
    for (int i = 0; i < personList.size(); i++) {
      Person p = personList.get(i);
      String s1 = p.getFirstName() + " " + p.getLastName();
      stringList.add(s1);
    }
    for (int i = 0; i < stringList.size(); i++) {
      for (int j = i; j < stringList.size(); j++) {
        if (stringList.get(i).compareToIgnoreCase(stringList.get(j)) > 0) {
          String aux = stringList.get(j);
          stringList.set(j, stringList.get(i));
          stringList.set(i, aux);
        }
      }
    }
    for (String s : stringList) System.out.println(s);
    return stringList;
  }
}
```

```java
public class Manager {
  public List<String> getPeopleNames(List<Person> people) {
    List<Person> personList = keepOnlyOveragePeople(people);

    ArrayList<String> stringList = new ArrayList<>();
    for (int i = 0; i < personList.size(); i++) {
      Person p = personList.get(i);
      String s1 = p.getFirstName() + " " + p.getLastName();
      stringList.add(s1);
    }
    for (int i = 0; i < stringList.size(); i++) {
      for (int j = i; j < stringList.size(); j++) {
        if (stringList.get(i).compareToIgnoreCase(stringList.get(j)) > 0) {
          String aux = stringList.get(j);
          stringList.set(j, stringList.get(i));
          stringList.set(i, aux);
        }
      }
    }
    for (String s : stringList) System.out.println(s);
    return stringList;
  }
}
```

```java
List<Person> overagePeople =
                    keepOnlyOveragePeople(people);
```

```java
public class Manager {
  public List<String> getPeopleNames(List<Person> people) {
    List<Person> overagePeople = keepOnlyOveragePeople(people);


    ArrayList<String> stringList = new ArrayList<>();
    for (int i = 0; i < personList.size(); i++) {
      Person p = personList.get(i);
      String s1 = p.getFirstName() + " " + p.getLastName();
      stringList.add(s1);
    }
    for (int i = 0; i < stringList.size(); i++) {
      for (int j = i; j < stringList.size(); j++) {
        if (stringList.get(i).compareToIgnoreCase(stringList.get(j)) > 0) {
          String aux = stringList.get(j);
          stringList.set(j, stringList.get(i));
          stringList.set(i, aux);
        }
      }
    }
    for (String s : stringList) System.out.println(s);
    return stringList;
  }
}
```

```java
public class Manager {
  public List<String> getPeopleNames(List<Person> people) {
    List<Person> overagePeople = keepOnlyOveragePeople(people);

    ArrayList<String> stringList = new ArrayList<>();
    for (int i = 0; i < personList.size(); i++) {
      Person p = personList.get(i);
      String s1 = p.getFirstName() + " " + p.getLastName();
      stringList.add(s1);
    }
    for (int i = 0; i < stringList.size(); i++) {
      for (int j = i; j < stringList.size(); j++) {
        if (stringList.get(i).compareToIgnoreCase(stringList.get(j)) > 0) {
          String aux = stringList.get(j);
          stringList.set(j, stringList.get(i));
          stringList.set(i, aux);
        }
      }
    }
    for (String s : stringList) System.out.println(s);
    return stringList;
  }
}
```

```java
List<String> stringList =
                    getPeopleFullNames(overagePeople);
```

```java
private ArrayList<String> getPeopleFullNames
                    (List<Person> overagePeople) {
  ArrayList<String> stringList = new ArrayList<>();

  for (Person s : overagePeople) {
    stringList.add(s.getFirstName() +
                                  " " + s.getLastName());
  }
  return stringList;
}
```

```java
public class Manager {
  public List<String> getPeopleNames(List<Person> people) {
    List<Person> personList = keepOnlyOveragePeople(people);

    List<String> stringList = getPeopleFullNames(overagePeople);

    for (int i = 0; i < stringList.size(); i++) {
      for (int j = i; j < stringList.size(); j++) {
        if (stringList.get(i).compareToIgnoreCase(stringList.get(j)) > 0) {
          String aux = stringList.get(j);
          stringList.set(j, stringList.get(i));
          stringList.set(i, aux);
        }
      }
    }
    for (String s : stringList) System.out.println(s);
    return stringList;
  }
}
```

```java
List<String> overagePeopleNames =
        getPeopleFullNames(overagePeople);
```

```java
public class Manager {
  public List<String> getPeopleNames(List<Person> people) {
    List<Person> overagePeople = keepOnlyOveragePeople(people);
    List<String> overagePeopleNames =
                                    getPeopleFullNames(overagePeople);

    for (int i = 0; i < stringList.size(); i++) {
      for (int j = i; j < stringList.size(); j++) {
        if (stringList.get(i).compareToIgnoreCase(stringList.get(j)) > 0) {
          String aux = stringList.get(j);
          stringList.set(j, stringList.get(i));
          stringList.set(i, aux);
        }
      }
    }

    for (String s : stringList) System.out.println(s);
    return stringList;
  }
}
```

```java
public class Manager {
  public List<String> getPeopleNames(List<Person> people) {
    List<Person> overagePeople = keepOnlyOveragePeople(people);
    List<String> overagePeopleNames =
                                    getPeopleFullNames(overagePeople);

    for (int i = 0; i < stringList.size(); i++) {
      for (int j = i; j < stringList.size(); j++) {
        if (stringList.get(i).compareToIgnoreCase(stringList.get(j)) > 0) {
          String aux = stringList.get(j);
          stringList.set(j, stringList.get(i));
          stringList.set(i, aux);
        }
      }
    }

    for (String s : stringList) System.out.println(s);
    return stringList;
  }
}
```

```java
overagePeopleNames =
        sortNamesAlphabetically(overagePeopleNames);
```

```java
private List<String> sortNamesAlphabetically(List<String> names) {
  List<String> sortedNames = new ArrayList<>(names);
  for (int i = 0; i < sortedNames.size(); i++)
    for (int j = i; j < sortedNames.size(); j++)
      if (sortedNames.get(i).compareToIgnoreCase(sortedNames.get(j)) > 0) {
        String aux = sortedNames.get(j);
        sortedNames.set(j, sortedNames.get(i));
        sortedNames.set(i, aux);
      }
  return sortedNames;
}
```

```java
public class Manager {
  public List<String> getPeopleNames(List<Person> people) {

    List<Person> overagePeople = keepOnlyOveragePeople(people);

    List<String> overagePeopleNames = getPeopleFullNames(overagePeople);

    overagePeopleNames = sortNamesAlphabetically(overagePeopleNames);

    for (String s : stringList) System.out.println(s);

    return stringList;
  }
}
```

```java
public class Manager {
  public List<String> getPeopleNames(List<Person> people) {

    List<Person> overagePeople = keepOnlyOveragePeople(people);

    List<String> overagePeopleNames = getPeopleFullNames(overagePeople);

    overagePeopleNames = sortNamesAlphabetically(overagePeopleNames);

    for (String s : stringList) System.out.println(s);

    return stringList;
  }
}
```

```java
printNames(overagePeopleNames);
```

```java
private void printNames(List<String> names) {
  for (String name : names)
    System.out.println(name);
}
```

```java
public class PersonService {

  public List<String> getOveragePeopleNames(List<Person> people) {

      List<Person> overagePeople = keepOnlyOveragePeople(people);

      List<String> overagePeopleNames = getPeopleFullNames(overagePeople);

      overagePeopleNames = sortNamesAlphabetically(overagePeopleNames);

      printNames(overagePeopleNames);

      return overagePeopleNames;
  }
...
}
```

```java
public class Person {
  private String firstName;
  private String lastName;
  private int age;
  private Status status;

  //constructor, getters, setters
}
```

```java
public enum Status {
  SINGLE,
  MARRIED,
  DIVORCED,
  WIDOW
}
```

```java
private ArrayList<String> getPeopleFullNames(List<Person> overagePeople) {
  ArrayList<String> stringList = new ArrayList<>();

  for (Person s : overagePeople) {
     stringList.add(s.getFirstName() + " " + s.getLastName());
  }
  return stringList;
}
```

```java
private ArrayList<String> getPeopleFullNames(List<Person> overagePeople) {
  ArrayList<String> stringList = new ArrayList<>();

  for (Person s : overagePeople) {
     stringList.add(s.getFirstName() + " " + s.getLastName());
  }
  return stringList;
}
```

```java
private ArrayList<String> getPeopleFullNames(List<Person> overagePeople) {
  ArrayList<String> stringList = new ArrayList<>();

  for (Person s : overagePeople) {
    stringList.add(s.getFirstName() + " " + s.getLastName());
  }
  return stringList;
}
```

```java
stringList.add(getFullName(s));
```

```java
private String getFullName(Person s) {
  return s.getFirstName() + " " + s.getLastName();
}
```

```java
private ArrayList<String> getPeopleFullNames(List<Person> overagePeople) {
  ArrayList<String> stringList = new ArrayList<>();

  for (Person s : overagePeople) {
    stringList.add(s.getFirstName() + " " + s.getLastName());
  }
  return stringList;
}
```

```java
stringList.add(getFullName(s));
```

```java
private String getFullName(Person s) {
  return s.getFirstName() + " " + s.getLastName();
}
```

```java
stringList.add(s.getFullName());
```

```java
public class Person {
  private String firstName;
  private String lastName;
  private int age;
  private Status status;

  //constructor, getters, setters

  String getFullName() {
    return getFirstName() + " " + getLastName();
  }
}
```

```java
public class PersonService {

  public List<String> getOveragePeopleNames(List<Person> people) {

    List<Person> overagePeople = keepOnlyOveragePeople(people);

    List<String> overagePeopleNames = getPeopleFullNames(overagePeople);

    overagePeopleNames = sortNamesAlphabetically(overagePeopleNames);

    printNames(overagePeopleNames);

    return overagePeopleNames;
  }
...
}
```

```java
public class Person {
  private String firstName;
  private String lastName;
  private int age;
  private Status status;

  //constructor, getters, setters

  String getFullName() {
    return getFirstName() + " " + getLastName();
  }
}
```